Análisis Problema Computable

1) **Análisis.**

a) **Evidencias que muestren la comprensión a fondo del problema y sus detalles**

Analizamos el mandato del ejercicio, pudimos darnos cuenta que se cumplía una sucesión aritmética para los espacios que se debían dar antes de los asteriscos y por igual una sucesión aritmética para la cantidad de asteriscos que deben imprimir por cada piso de la pirámide, el ejercicio nos indica que la cantidad de pisos que debe tener la pirámide debe ser ingresado por el usuario, asi que sabemos que para solucionar este ejercicio tenemos que realizar varios ciclos hasta que se cumplan condiciones relacionadas con el numero ingresado por el usuario. A continuación, los detalles del ejercicio:

- Se solicita una entrada al usuario, que indica la cantidad de pisos que debe tener la pirámide.
- Se debe validar la entrada, que no sea menor que cero, ni igual a cero, para que vale pueda cumplir una impresión adecuada.
- Se debe poner un parámetro a la entrada para no exceder una impresión mayor de los pisos que se pueden visualizar en la ventana a nivel horizontal.
- Se debe realizar bucles para poder cumplir las condiciones de la impresión, un bucle que me ayude a cumplir la secuencia de los pisos, que contenga otro bucle que permita cumplir la secuencia de los espacios en blancos y al finalizar ese bucle entre en otro bucle que permita imprimir los asteriscos siempre que se cumpla la sucesión. Al finalizar este bucle debe cambiar de piso y repetir todo nuevamente, hasta lograr imprimir todos los pisos suministrados por el usuario.
- Las impresiones deben ser claras y legibles, cada vez que se cumpla el ciclo de los espacios debe seguir la secuencia de la impresión de los asteriscos y hacer el cambio de piso para que el resultado de este ejercicio sea una pirámide que cumpla todos los parámetros deseados.
- El programa debe tener un fin, si no se cumplen condiciones establecidas debe finalizar el programa, asi como si se cumple todos los procesos deseados, alcanzando imprimir la pirámide completo.

b) **Identificar los elementos de información dados en los problemas que se puedan ser útiles para obtener la solución del problema planteado.**

- **Número de pisos deseado para la pirámide:** Este es el valor principal que el usuario debe proporcionar al programa. Representa la altura de la pirámide y es esencial para determinar cuántas filas de asteriscos se imprimirán.
- **Validación de entrada:** Además de conocer el número de pisos, es importante validar que la entrada del usuario sea adecuada. El programa necesita verificar que el número de pisos no sea 0 y 60, ya que una pirámide de 0 y 60 pisos no tiene sentido en este contexto. También debe asegurarse de que el número no sea excesivamente grande para evitar problemas en la impresión.

c) **Resolver MANUALMENTE el problema (sin tomar en cuenta aspectos computacionales). La solución MANUAL DEL PROBLEMA debe ser definida con el máximo detalle**

Iniciar con un número de pisos: El usuario debe proporcionar un número que representará la altura de la pirámide. Supongamos que el usuario ingresa 5.

Dibujo de la pirámide: Ahora, dibujemos la pirámide de asteriscos paso a paso, siguiendo estos pasos:

Paso 1: La primera fila debe tener un solo asterisco. Para centrarla, dejamos algunos espacios en blanco al principio de la línea. En este caso, dejamos cuatro espacios en blanco (5 - 1 = 4) y luego colocamos el asterisco. La fila 1 se ve así: " *".
Paso 2: La segunda fila tendrá tres asteriscos. Dejamos tres espacios en blanco al principio de la línea (5 - 2 = 3) y colocamos los tres asteriscos. La fila 2 se ve así: " ***".
Paso 3: La tercera fila tendrá cinco asteriscos. Dejamos dos espacios en blanco al principio de la línea (5 - 3 = 2) y colocamos los cinco asteriscos. La fila 3 se ve así: " *****".
Paso 4: La cuarta fila tendrá siete asteriscos. Dejamos un espacio en blanco al principio de la línea (5 - 4 = 1) y colocamos los siete asteriscos. La fila 4 se ve así: " *******".
Paso 5: La quinta fila tendrá nueve asteriscos. No dejamos espacio en blanco (5 - 5 = 0) y colocamos los nueve asteriscos. La fila 5 se ve así: "*********".
Finalizar la pirámide: Hemos completado la impresión de la pirámide de asteriscos.

2) **Diseño.**

a) **Justificar por qué diseñaron esa solución para el algoritmo para solucionar ese enunciado (Por que usted entiende que esa solución resuelve el problema)**

- **Sencillez y claridad:** El diseño de la solución es sencillo y claro. Se utiliza un bucle for para iterar a través de cada fila de la pirámide y calcular la cantidad adecuada de espacios en blanco y asteriscos. Esto hace que el código sea fácil de entender y mantener.
- **Control de entrada:** El algoritmo incorpora una validación de entrada para garantizar que el número de pisos ingresado sea válido. Esto es esencial para evitar situaciones de error, como una pirámide de 0 pisos o números excesivamente grandes. La validación agrega robustez al programa.
- **Secuencial:** El algoritmo es secuencial y puede manejar diferentes alturas de pirámide simplemente cambiando el número de pisos. No es necesario modificar el código principal, lo que facilita su adaptación a diferentes necesidades.
- **Claridad en el proceso manual:** El proceso manual para la solución es fácil de seguir y entender. Cada paso se detalla claramente, lo que facilita la implementación manual de la solución sin necesidad de conocimientos informáticos.
- **Resultado deseado:** La solución produce el resultado deseado: una pirámide de asteriscos con la altura especificada por el usuario. Esto se logra al calcular cuidadosamente la posición de los asteriscos y los espacios en blanco en cada fila.

b) **Justificar por qué la solución cumple con CADA UNA de las "Características de los algoritmos"?**

- **Finitud:** El algoritmo es finito en el sentido de que tiene un número finito de pasos, que está determinado por el valor ingresado por el usuario, es decir, el número de pisos de la pirámide. El bucle for que imprime la pirámide iterará exactamente una cantidad veces, donde una cantidad veces es el número de pisos ingresado, y el programa finalizará después de eso.
- **Entrada:** El algoritmo toma entrada del usuario en forma de un número entero que representa la cantidad de pisos deseada de la pirámide. Además, incorpora validación de entrada para asegurarse de que la entrada cumple con ciertas restricciones (no puede ser 0 y debe ser menor o igual a 60).
- **Definición:** El algoritmo está claramente definido. Utiliza un bucle for para imprimir cada fila de la pirámide, calculando la cantidad adecuada de espacios en blanco y asteriscos en cada piso. También define claramente las condiciones para la validación de la entrada y cómo manejar los casos de error.

- **Salida:** El algoritmo produce una salida clara y legible que cumple con el objetivo del problema. Una pirámide de asteriscos cumpliendo de manera efectiva los pasos mencionados anteriormente para así no cometer ningún tipo de equivocación.
- **Claridad:** El algoritmo es claro y fácil de entender. Utiliza un bucle for a través de los pisos de la pirámide, y cada paso se describe detalladamente en el proceso manual. Además, se comunican mensajes de error claros en caso de entrada incorrecta.
- **Efectividad:** La solución es efectiva en el sentido de que cumple con el propósito del problema de imprimir una pirámide de asteriscos. Incorpora validaciones para prevenir errores comunes, como una entrada de 0 o números excesivamente grandes, lo que mejora la efectividad del programa.

3) Codificación (implementación).

a) Justificar los statements usados.

Se incluye la biblioteca iostream (#include <iostream>) para habilitar las operaciones de entrada y salida estándar. Se utiliza using namespace std; para evitar la necesidad de anteponer std:: a las funciones y objetos de la biblioteca estándar de C++. Se declara una variable int numero; para almacenar el número de pisos de la pirámide que el usuario proporcionará. El mensaje "Ingrese el número de pisos para la pirámide (de 1 a 60): " se muestra al usuario para solicitar la entrada del número de pisos. El valor ingresado por el usuario se almacena en la variable número usando cin >> numero; Luego, se utiliza una estructura condicional if-else para validar el valor ingresado: Si número es menor o igual a cero (numero <= 0), se imprime el mensaje de error "¡Error! - El número de pisos no puede ser menor o igual que 0." Si número es mayor que 60 (numero > 60), se imprime el mensaje de error "¡Error! El número de pisos no puede ser mayor que 60." Si el número ingresado no es un valor inválido (menor o igual a cero o mayor que 60), se procede a dibujar la pirámide. Un bucle for se utiliza para iterar a través de cada piso de la pirámide (int piso = 1; piso <= numero; piso++). Otro bucle for se utiliza para imprimir espacios en blanco antes de los asteriscos. La cantidad de espacios en blanco disminuye a medida que aumenta el número de pisos (espacio < numero - piso). Un tercer bucle for se utiliza para imprimir asteriscos. La cantidad de asteriscos aumenta a medida que aumenta el número de pisos (2 * piso - 1). Se imprime un salto de línea (endl) para comenzar una nueva línea y pasar al siguiente piso de la pirámide.

b) Justificar el uso de variables: acumuladores, contadores, etc.

int numero: Esta variable almacena el número de pisos que el usuario ingresa. No es un acumulador ni un contador, pero es fundamental para determinar el número de filas que se deben generar en la pirámide.
int piso: En el bucle for, se utiliza la variable piso para rastrear el número actual de piso en la pirámide. A medida que el bucle se ejecuta, piso se incrementa de 1 en 1. Aunque no es un contador en el sentido tradicional, controla la generación de los diferentes pisos.
int espacio: En el segundo bucle for, se utiliza la variable espacio para controlar la cantidad de espacios en blanco que se imprimen antes de los asteriscos en cada fila. A medida que aumenta piso, la cantidad de espacios se reduce. Aunque no se llama "contador," cumple una función similar al rastrear el número de espacios.
int asterisco: En el tercer bucle for, se utiliza la variable asterisco para controlar la cantidad de asteriscos que se imprimen en cada fila. A medida que piso aumenta, se imprimen más asteriscos. Al igual que espacio, no se llama "contador" pero desempeña una función similar.

c) Comprobar el módulo (realizar corrida de escritorio utilizando como base el CODIGO).

cout	cin	piso	numero	espacio	asterisco
"Ingrese el numero de pisos para la piramide (de 1 a 60): "					
	3		3		
		1			
				0	
				0<3-1=2	
" "					
				1	
				1<3-1=2	
" "					
				2	
				1<3-2=1	
					0
					0<2*1-1=1
"*"					
					1
					1<2*1-1=1
		2			
				0	
				0<3-2=1	
" "					
				1	
				1<3-2=1	
					0
					0<2*2-1=3
"*"					
					1
					1<2*2-1=3
"*"					
					2
					2<2*2-1=3
"*"					
					3
					3<2*2-1=3
		3			
				0	
				0<3-3=0	
					0
					0<2*3-1=5
"*"					
					1
					1<2*3-1=5
"*"					
					2
					2<2*3-1=5
"*"					
					3
					3<2*3-1=5
"*"					
					4
					4<2*3-1=5
"*"					
					5
					5<2*3-1=5

```
  *
 ***
*****
```